

turtlebot3_project3/Part01/proj3p2_FNU_aryan_keyur.py

```

1  # ENPM 661: Planning for Robotics
2  # Project 3 - Phase 2
3  # authors: Keyur Borad, Aryan Mishra, FNU Koustubh
4  # This script is used to implement the A* algorithm to find the path from start to goal position
5  # The script also generates a video of the path found by the algorithm
6  #github link: https://github.com/koustubh1012/a_star_planner_ros2
7
8
9  # import libraries
10
11  import cv2                                # import the OpenCV library
12  import numpy as np                        # import the numpy library
13  import heapq as hq                       # import the heapq library
14  import math                              # import the math library
15  import time                              # import the time library
16
17  canvas = np.ones((200,600,3))            # creating a frame for video generation
18  obstacle_set = set()                     # set to store the obstacle points
19  obstacle_list = []                       # list to store the obstacle points in order for videp
20
21  c2c_node_grid = [[float('inf')] * 200 for _ in range(600)]      # create a 2D array for storing cost to come
22  tc_node_grid = [[float('inf')] * 200 for _ in range(600)]        # create a 2D array for storing cost to come
23  closed_set = set()                                                 # set to store the value of visited and closed points
24  closed_list = []
25  visited={}
26
27  C = int(input("Enter the clearance from the obstacle in mm: "))    # Get clearance from the user
28  C = C/10
29
30  # All units are in cm
31  R = 66/20                                                            # Robot wheel radius
32  r = 22                                                              # Robot radius
33  L = 28.7                                                            # Robot wheel track
34  T = C + r                                                            # Total clearance
35
36  t_max = 3.5
37  t_min = 0.1
38
39  x_goal = 0 # Initialize the goal x coordinate

```

```

40 y_goal = 0 # Initialize the goal y coordinate
41 x_start = 0 # Initialize the start x coordinate
42 y_start = 0 # Initialize the start y coordinate
43 node=(0, 0, 1, [], (x_start, y_start), 0)
44
45
46
47 '''
48 Loop to define the obstacle points in the map
49 '''
50 for y in range(200):
51     for x in range(600):
52         canvas[y,x] = [255,255,255]
53         if (0<=y<=T):
54             obstacle_set.add((x,y))
55             obstacle_list.append((x,y))
56             c2c_node_grid[x][y] = -1
57             tc_node_grid[x][y] = -1
58         elif (0<=x<=T):
59             obstacle_set.add((x,y))
60             obstacle_list.append((x,y))
61             c2c_node_grid[x][y] = -1
62             tc_node_grid[x][y] = -1
63         elif (200-T<=y<200):
64             obstacle_set.add((x,y))
65             obstacle_list.append((x,y))
66             c2c_node_grid[x][y] = -1
67             tc_node_grid[x][y] = -1
68         elif (600-T<=x<600):
69             obstacle_set.add((x,y))
70             obstacle_list.append((x,y))
71             c2c_node_grid[x][y] = -1
72             tc_node_grid[x][y] = -1
73
74         elif (150-T<=x<=175+T) and (100-T<=y<=200):
75             obstacle_set.add((x,y))
76             obstacle_list.append((x,y))
77             c2c_node_grid[x][y] = -1
78             tc_node_grid[x][y] = -1
79
80         elif (250-T<=x<=275+T) and (0<=y<=100+T):
81             obstacle_set.add((x,y))

```

```

# loop to define the obstacle points : x
# loop to define the obstacle points : y
# mark the points in the frame with white color
# points in the bottom boundary
# add the points to the obstacle set
# add the points to the obstacle list
# mark the points in the cost to come grid with -1
# mark the points in the total cost grid with -1
# points in the left boundary
# add the points to the obstacle set
# add the points to the obstacle list
# mark the points in the cost to come grid with -1
# mark the points in the total cost grid with -1
# points in the top boundary
# add the points to the obstacle set
# add the points to the obstacle list
# mark the points in the cost to come grid with -1
# mark the points in the total cost grid with -1
# points in the right boundary
# add the points to the obstacle set
# add the points to the obstacle list
# mark the points in the cost to come grid with -1
# mark the points in the total cost grid with -1

# points in first rectangle
# add the points to the obstacle set
# add the points to the obstacle list
# mark the points in the cost to come grid with -1
# mark the points in the total cost grid with -1

# points in second rectangle
# add the points to the obstacle set

```

```

82         obstacle_list.append((x,y))           # add the points to the obstacle list
83         c2c_node_grid[x][y] = -1              # mark the points in the cost to come grid with -1
84         tc_node_grid[x][y] = -1               # mark the points in the total cost grid with -1
85
86         # Points in the Circle shaped obstacle
87         elif ((x-420)**2 + (y-120)**2 <= (60+T)**2): # points in the first rectangle of Concave shaped
obstacle
88             obstacle_set.add((x,y))           # add the points to the obstacle set
89             obstacle_list.append((x,y))       # add the points to the obstacle list
90             c2c_node_grid[x][y] = -1          # mark the points in the cost to come grid with -1
91             tc_node_grid[x][y] = -1           # mark the points in the total cost grid with -1
92
93     # Mark the obstacle points in the frame, including points after bloating
94     for point in obstacle_list:               # loop to mark the obstacle points
95         canvas[point[1],point[0]] = [255, 0, 0] # mark the obstacle points with blue color
96
97     # Draw the obstacles in the frame, excluding the points after bloating
98     cv2.rectangle(canvas, (150, 200), (175, 100), (0 , 0, 255), -1) # draw the first rectangle
99     cv2.rectangle(canvas, (250, 100), (275, 0), (0 , 0, 255), -1)   # draw the second rectangle
100    cv2.circle(canvas,(420, 120), 60, (0,0,255),-1) # draw the circle shaped obstacle
101
102    valid_start = False # flag to check if the start point
is valid
103
104    while not valid_start: # loop to check if the start point
is valid
105        start = input("Enter the start coordinates and orientation as (x, y, theta): ") #
get the start coordinate and orientaion from the user
106        [x_start, y_start, theta_start] = [int(i) for i in start.split()]
107        x_start = int(x_start/10) + 50
108        y_start = int(y_start/10) + 100
109        if (x_start, y_start) in obstacle_set: # check if the start point is in
the obstacle set
110            print("Invalid coordinates, Enter again") # print error message
111        else:
112            initial_node = (0, 0, 1, [], (x_start, y_start), theta_start) # create the initial node
113            valid_start = True # set the flag to true
114
115    valid_goal = False # flag to check if the goal point
is valid
116
117    while not valid_goal:
118        goal = input("Enter the goal coordinates as (x, y): ") # get the start coordinate
and orientaion from the user

```

```

119     [x_goal, y_goal] = [int(i) for i in goal.split()]
120     x_goal = int(x_goal/10) + 50
121     y_goal = int(y_goal/10) + 100
122     if (x_goal, y_goal) in obstacle_set:                                # check if the goal point is in the
obstacle set                                                                # print error message
123         print("Invalid coordinates, Enter again")
124     else:
125         goal = (x_goal, y_goal)                                           # create the goal node
126         valid_goal = True                                                # set the flag to true
127
128     valid_rpm = False
129
130     while not valid_rpm:
131         rpm = input("Enter the RPM1 and RPM2: ")
132         [rpm1, rpm2] = [int(i) for i in rpm.split()]
133         if (5<=rpm1<=75 and 5<=rpm2<=75):
134             valid_rpm = True
135         else:
136             print("Invalid rpm, Enter again in the range of 5 and 75")
137
138     min_rpm = min(rpm1, rpm2)
139
140     t = round(((t_max - t_min)*(min_rpm - 75)/(5 - 75)) + t_min, 2)      # Calculate time step
141     print("Calculated time step: ", t)
142
143     # Funtion to update the visted nodes
144     def visited_node(node):
145         visited.update({node[2]:node[4]})
146
147     def actionnnn(node, rpm1, rpm2):
148         ul = 2*math.pi*rpm1/60                                           # calculate the left wheel velocity
149         ur = 2*math.pi*rpm2/60                                           # calculate the right wheel velocity
150         new_heading = (node[5] + np.rad2deg(((R/L)*(ul - ur)*t))) % 360  # get the current heading of the robot
151         x_vel = (R/2)*(ur+ul)*np.cos(np.deg2rad(new_heading))             # calculate the x velocity
152         y_vel = (R/2)*(ur+ul)*np.sin(np.deg2rad(new_heading))            # calculate the y velocity
153         # print("X vel: ", x_vel)
154         # print("Y vel: ", y_vel)
155         x = node[4][0] + x_vel*t                                           # calculate the new x coordinate
156         y = node[4][1] + y_vel*t                                           # calculate the new y coordinate
157         x = round(x)
158         y = round(y)
159         c2c = node[1] + math.sqrt((x_vel*t)**2 + (y_vel*t)**2)           # calculate the cost to come

```

```

160     c2g = math.sqrt((y_goal-y)**2 + (x_goal-x)**2) # calculate the cost to goal
161     tc = c2c + c2g # calculate the total cost
162     return (x,y),new_heading,tc,c2c # return the new node's coordinates, heading, total cost and
cost to come
163
164 action_lists=[(0,rpm1),(rpm1,0),(rpm1,rpm1),(rpm1,rpm2),(rpm2,rpm1),(0,rpm2),(rpm2,0),(rpm2,rpm2)] # list of
actions
165
166
167 start_time = time.time() # get the start time of the program
168 new_index = 1 # initialize the index
169 open_list = [] # initialize the open list
170 hq.heappush(open_list,initial_node) # Push initial node to the list
171 hq.heapify(open_list) # covers list to heapq data type
172
173 while(open_list):
174     node = hq.heappop(open_list) # pop the node with lowest cost to come
175     closed_list.append(node[4]) # add the node coordinates to closed set
176     closed_set.add(node[4]) # add the node coordinates to closed set
177     visited_node(node) # add the node to the visited list
178     index = node[2] # store the index of the current node
179     parent_index = node[3] # store the parent index list of current node
180
181     node_dist = math.sqrt((node[4][0]-x_goal)**2 + (node[4][1]-y_goal)**2) # calculate the distance between the
current node and goal node
182     if node_dist < 5: # if the node is goal position, exit the loop
183         print("Goal reached")
184         break
185
186     for action_set in action_lists: # loop to get the new node's
coordinates, heading, total cost and cost to come
187         point, new_heading, tc, c2c = actionnn(node,action_set[0],action_set[1])
# get the new node's coordinates, heading, total cost and cost to come
188         if point not in obstacle_set and point not in closed_set and 0<=point[0]<600 and 0<=point[1]<200:
# check if the new node is in the obstacle set or visited list
189             x = int(point[0]) # get the x coordinate of the new
node
190             y = int(point[1]) # get the y coordinate of the new
node
191             try:
192                 if tc<tc_node_grid[x][y]: # check if the new cost to come is
less than original cost to come
193                     new_parent_index = parent_index.copy() # copy the parent index list of the
current node

```

```

194         new_parent_index.append(index)                # Append the current node's index
to the new node's parent_index_list
195         new_index+=1                                # increment the index
196         tc_node_grid[x][y] = tc                      # Update the new total cost
197         c2c_node_grid[x][y] = c2c                    # Update the new cost to come
198         new_node = (tc, c2c, new_index, new_parent_index, (x,y), new_heading) # create the new node
199         hq.heappush(open_list, new_node)              # push the new node to the open
list
200     except:
201         print(x,y)
202
203
204
205     print("Actual goal reached :", (node[4][0]-50)*10, (node[4][1]-100)*10)    # print the actual goal position
206
207
208     path = node[3]                # Get the parent node list
209     counter = 0                   # counter to count the frames to write on video
210
211     fourcc = cv2.VideoWriter_fourcc(*'mp4v') # Codec for MP4 format
212     video_writer = cv2.VideoWriter('output/new_output.mp4', fourcc, 60, (600, 200)) # Video writer object
213
214     '''
215     Loop to mark the explored nodes in order on the frame
216     '''
217     print("Exploring map")
218
219
220     # print(closed_set)
221     for node in closed_list:      # loop to mark the explored nodes
222
223         # canvas[node[1], node[0]] = [0, 255, 0]          # mark the explored nodes with green
color
224         cv2.circle(canvas, node, 1, (0,255,0), -1)      # mark the goal point with red color
225         counter +=1                                       # increment the counter
226         if counter%25 == 0 or counter == 0:              # check if the counter is divisible by
500
227             cv2.circle(canvas,(x_start, y_start), 5, (0,0,255), -1) # mark the goal point with red color
228             cv2.circle(canvas,(x_goal, y_goal), 5, (255,0,255), -1) # mark the goal point with red color
229             canvas_flipped = cv2.flip(canvas,0)          # flip the frame
230             canvas_flipped_uint8 = cv2.convertScaleAbs(canvas_flipped) # convert the frame to uint8
231             # cv2.imshow('window', canvas_flipped_uint8)
232             # cv2.waitKey(1)

```

```
233     video_writer.write(canvas_flipped_uint8)                # write the frame to video
234
235     '''
236     Loop to mark the path created
237     '''
238     print("Backtracking")
239
240     for index in path:                                       # loop to mark the path
241         coord=visited[index]                                # get the coordinates of the node
242
243         cv2.circle(canvas, (coord[0],coord[1]), 1, [0,0,0], -1) # mark the path with black color
244         canvas_flipped = cv2.flip(canvas, 0)                # flip the frame
245         canvas_flipped_uint8 = cv2.convertScaleAbs(canvas_flipped) # convert the frame to uint8
246         # cv2.imshow('window',canvas_flipped_uint8)
247         # cv2.waitKey(1)
248
249         video_writer.write(canvas_flipped_uint8)            # write the frame to video
250
251
252
253     '''
254     Loop to add some additional frames at the end of the video
255     '''
256     for i in range(150):
257         video_writer.write(canvas_flipped_uint8)            # write the frame to video
258
259     print("Video Processed")                                # print message
260
261     video_writer.release()                                   # release the video writer
262
263     end_time = time.time()                                   # get the end time of the program
264     print(f"The runtime of my program is {end_time - start_time} seconds.") # print the runtime of the program
265
266     cv2.imshow("canvas", canvas_flipped_uint8)              # show the final frame
267     cv2.waitKey(0)                                           # wait for the key press
268     cv2.destroyAllWindows()                                  # close the window
```

turtlebot3_project3/Part02/turtlebot3_projet3/scripts/a_star_closed_loop.py

```

14 # ENPM 661: Planning for Robotics
15 # Project 3 - Phase 2
16 # authors: Keyur Borad, Aryan Mishra, FNU Koustubh
17 # This script is used to implement the A* algorithm for the Turtlebot3 robot to navigate in a known environment
18 #github link:https://github.com/koustubh1012/a_star_planner_ros2
19
20 #!/usr/bin/env python3
21
22 import rclpy                                     # Import the ROS client library for
Python                                           # Import the Node class from the
23 from rclpy.node import Node                     # Import the Twist message from the
rclpy library                                   # Import the Odometry message from
24 from geometry_msgs.msg import Twist            # Import the keyboard module from
geometry_msgs package                          # Import the numpy library
25 from nav_msgs.msg import Odometry              # Import the heapq library
the nav_msgs package                          # Import the math library
26 from pynput import keyboard
the pynput library
27 import numpy as np
28 import heapq as hq
29 import math
30
31 # Define key codes
32
33 class AStarControllerNode(Node):
34     """
35     AStarControllerNode class is a subclass of the Node class
36
37     Args:
38         Node (Class): The Node class from the rclpy library
39         subscription (Subscription): The subscription object to subscribe to the /odom topic
40         cmd_vel_pub (Publisher): The publisher object to publish to the /cmd_vel topic
41         velocity_msg (Twist): The Twist message object to publish the velocity commands
42         waypoints (list): The list to store the waypoints of the path
43         i (int): The index to iterate through the waypoints
44         x_goal (int): The x coordinate of the goal point
45         y_goal (int): The y coordinate of the goal point
46         x_start (int): The x coordinate of the start point
47         y_start (int): The y coordinate of the start point
48         C (int): The clearance value

```

```

49     rpm1 (int): The rpm of the left wheel
50     rpm2 (int): The rpm of the right wheel
51     cmd_vel_pub (Publisher): The publisher object to publish to the /cmd_vel topic
52     velocity_msg (Twist): The Twist message object to publish the velocity commands
53     """
54
55     def __init__(self):
56         super().__init__('a_star_controller_node')
57         self.subscription = self.create_subscription(Odometry, '/odom', self.odom_callback, 10)
58         self.subscription # prevent unused variable warning
59
60         self.declare_parameter('x_pose', 0.0) # declare the parameter x_pose,
y_pose, goal_x, goal_y, clearance, rpm1 and rpm2
61         self.declare_parameter('y_pose', 0.0)
62         self.declare_parameter('goal_x', 5000.0)
63         self.declare_parameter('goal_y', 0.0)
64         self.declare_parameter('clearance', 50.0)
65         self.declare_parameter('rpm1', 20.0)
66         self.declare_parameter('rpm2', 30.0)
67
68         self.x_goal = int(self.get_parameter('goal_x').value/10) + 50 # get the goal x and y coordinates
and converting to cm
69         self.y_goal = int(self.get_parameter('goal_y').value/10) + 100
70         self.x_start = int(self.get_parameter('x_pose').value/10) + 50 # get the start x and y coordinates
and converting to cm
71         self.y_start = int(self.get_parameter('y_pose').value/10) + 100
72         self.C = self.get_parameter('clearance').value/10 # get the clearance value and
converting to cm
73         self.rpm1 = self.get_parameter('rpm1').value # get the rpm1 and rpm2 values
74         self.rpm2 = self.get_parameter('rpm2').value
75
76
77         self.get_logger().info('A start controller node initialised') # print the message that the node is
initialised
78
79         self.a_star_solver() # call the a_star_solver function
80
81         self.i = 0 # set the index to 0
82         self.get_logger().info('Creating Publisher') # print the message that the
publisher is being created
83         self.cmd_vel_pub = self.create_publisher(Twist, '/cmd_vel', 10) # create the publisher object to
publish to the /cmd_vel topic
84         self.velocity_msg = Twist() # create the Twist message object to
publish the velocity commands

```

```

85
86
87     def odom_callback(self, msg):
88         """
89         odom_callback function is the callback function for the /odom topic
90
91         Args:
92             msg (Odometry): The Odometry message from the /odom topic
93             velocity_msg (Twist): The Twist message object to publish the velocity commands
94         """
95         if (self.i >= len(self.waypoints)):
96             self.velocity_msg.linear.x = 0.0 # set the linear velocity
97             self.velocity_msg.angular.z = 0.0 # set the angular velocity
98             self.cmd_vel_pub.publish(self.velocity_msg) # publish the velocity
99             self.get_logger().info('Goal reached', once=True) # print the goal reached
100         else:
101             max_rpm = max(self.rpm1, self.rpm2) # get the maximum rpm
102             x = msg.pose.pose.orientation.x # get the x,y,z and w
103             y = msg.pose.pose.orientation.y
104             z = msg.pose.pose.orientation.z
105             w = msg.pose.pose.orientation.w
106             yaw = math.atan2(2 * (w * z + x * y), 1 - 2 * (y*y + z*z)) # convert quaternion to
107             robot_x = msg.pose.pose.position.x # get the x and y
108             robot_y = msg.pose.pose.position.y
109             wpt_x = self.waypoints[self.i][0] # get the x and y
110             wpt_y = self.waypoints[self.i][1]
111             dist = math.sqrt((robot_x-wpt_x)**2 + (robot_y-wpt_y)**2) # calculating the distance
112             yaw_req = math.atan2(wpt_y-robot_y, wpt_x-robot_x) # calculating the required
113             e = yaw_req - yaw
114             """
115             max angular velocity of the robot cannot exceed 1.82 rad/s
116             max linear velocity of the robot cannot exceed 0.22 m/s
117             """
118             if dist>0.15:
119                 self.velocity_msg.angular.z = 0.4*e # set the angular velocity

```

```

120         if self.velocity_msg.angular.z > 1.82:           # check if the angular velocity is
greater than the maximum angular velocity
121             self.velocity_msg.angular.z = 1.82           # set the angular velocity to the
maximum angular velocity
122         elif self.velocity_msg.angular.z < -1.82:         # check if the angular velocity is
less than the minimum angular velocity
123             self.velocity_msg.angular.z = -1.82          # set the angular velocity to the
minimum angular velocity
124
125             self.velocity_msg.linear.x = (2*math.pi*max_rpm/60)*33/1000
126             self.cmd_vel_pub.publish(self.velocity_msg)
127         else:
128             self.get_logger().info('Next Waypoint: %s %s' % (self.waypoints[self.i][0], self.waypoints[self.i]
[1]))
129             self.i += 1
130
131
132     def a_star_solver(self):
133
134         obstacle_set = set()                               # set to store the obstacle points
135         obstacle_list = []                                 # list to store the obstacle points in order for videp
136         c2c_node_grid = [[float('inf')] * 200 for _ in range(600)] # create a 2D array for storing cost to
come
137         tc_node_grid = [[float('inf')] * 200 for _ in range(600)] # create a 2D array for storing cost to
come
138         closed_set = set()                                 # set to store the value of visited and closed points
139         closed_list = []
140         visited={}
141
142         # All units are in cm
143         R = 66/20                                           # Robot wheel radius
144         r = 22                                              # Robot radius
145         L = 28.7                                           # Robot wheel track
146         T = self.C + r                                     # Total clearance
147
148         t_max = 3.5
149         t_min = 0.1
150
151         theta_start = 0.0
152
153         node=(0, 0, 1, [], (self.x_start, self.y_start), 0)
154         initial_node = (0, 0, 1, [], (self.x_start, self.y_start), theta_start, []) # create the initial node
155         ...
156

```

```

157     Loop to define the obstacle points in the map
158     ...
159     for y in range(200):
160         for x in range(600):
161             if (0<=y<=T):
162                 obstacle_set.add((x,y))
163                 obstacle_list.append((x,y))
164                 c2c_node_grid[x][y] = -1
-1
165                 tc_node_grid[x][y] = -1
166             elif (0<=x<=T):
167                 obstacle_set.add((x,y))
168                 obstacle_list.append((x,y))
169                 c2c_node_grid[x][y] = -1
-1
170                 tc_node_grid[x][y] = -1
171             elif (200-T<=y<200):
172                 obstacle_set.add((x,y))
173                 obstacle_list.append((x,y))
174                 c2c_node_grid[x][y] = -1
-1
175                 tc_node_grid[x][y] = -1
176             elif (600-T<=x<600):
177                 obstacle_set.add((x,y))
178                 obstacle_list.append((x,y))
179                 c2c_node_grid[x][y] = -1
-1
180                 tc_node_grid[x][y] = -1
181
182             elif (150-T<=x<=175+T) and (100-T<=y<=200):
183                 obstacle_set.add((x,y))
184                 obstacle_list.append((x,y))
185                 c2c_node_grid[x][y] = -1
-1
186                 tc_node_grid[x][y] = -1
187
188             elif (250-T<=x<=275+T) and (0<=y<=100+T):
189                 obstacle_set.add((x,y))
190                 obstacle_list.append((x,y))
191                 c2c_node_grid[x][y] = -1
-1
192                 tc_node_grid[x][y] = -1
193
194             # Points in the Circle shaped obstacle

```

```

195         elif ((x-420)**2 + (y-120)**2 <= (60+T)**2): # points in the first rectangle of Concave shaped
obstacle
196             obstacle_set.add((x,y)) # add the points to the obstacle set
197             obstacle_list.append((x,y)) # add the points to the obstacle list
198             c2c_node_grid[x][y] = -1 # mark the points in the cost to come grid with
-1
199             tc_node_grid[x][y] = -1 # mark the points in the total cost grid with -1
200
201         min_rpm = min(self.rpm1, self.rpm2)
202
203         valid_goal = False
204
205         while not valid_goal:
206             goal = input("Enter the goal coordinates as (x, y) in mm: ") # get the start
coordinate and orientaion from the user
207             [self.x_goal, self.y_goal] = [int(i) for i in goal.split()] # split the input
and store the x and y coordinates
208             self.x_goal = int(self.x_goal/10) + 50 # convert the x
coordinate to cm
209             self.y_goal = int(self.y_goal/10) + 100 # convert the y
coordinate to cm
210             if (self.x_goal, self.y_goal) in obstacle_set: # check if the
goal point is in the obstacle set
211                 self.get_logger().error("Invalid coordinates, Enter again") # print error
message
212             else:
213                 valid_goal = True # set the flag to
true
214
215             self.t = round(((t_max - t_min)*(min_rpm - 75)/(5 - 75)) + t_min, 2) # Calculate time
step
216
217             def visited_node(node): # function to add
the visited node to the visited list
218                 visited.update({node[2]:node[4]})
219
220             def actionnn(node, rpm1, rpm2):
221                 ul = 2*math.pi*rpm1/60 # calculating the left wheel
velocity
222                 ur = 2*math.pi*rpm2/60 # calculating the right wheel
velocity
223                 theta_dot = (R/L)*(ul - ur)
224                 new_heading = (node[5] + (np.rad2deg(theta_dot)*self.t)) % 360 # get the current heading of the
robot
225                 x_vel = (R/2)*(ur+ul)*np.cos(np.deg2rad(new_heading)) # calculating the x velocity

```

```

226         y_vel = (R/2)*(ur+ul)*np.sin(np.deg2rad(new_heading))           # calculating the y velocity
227         x = node[4][0] + x_vel*self.t # calculating the new x coordinate
228         y = node[4][1] + y_vel*self.t # calculating the new y coordinate
229         x = round(x)
230         y = round(y)
231         c2c = node[1] + math.sqrt((x_vel*self.t)**2 + (y_vel*self.t)**2)   #
calculate the cost to come
232         c2g = math.sqrt((self.y_goal-y)**2 + (self.x_goal-x)**2) # calculate the cost to goal
233         tc = c2c + c2g # calculate the total cost
234         return (x,y),new_heading,tc,c2c # return the new node's coordinates, heading, total cost
and cost to come

235
236         action_lists=[(0,self.rpm1),(self.rpm1,0),(self.rpm1,self.rpm1),(self.rpm1,self.rpm2),
237                       (self.rpm2,self.rpm1),(0,self.rpm2),(self.rpm2,0),(self.rpm2,self.rpm2)]
238
239         new_index = 1
240         open_list = []
241         hq.heappush(open_list,initial_node) # Push initial node to the list
242         hq.heapify(open_list) # covers list to heapq data type
243
244         while(open_list):
245             node = hq.heappop(open_list) # pop the node with lowest cost to come
246             closed_list.append(node[4]) # add the node coordinates to closed set
247             closed_set.add(node[4])
248             visited_node(node) # add the node to the visited list
249             index = node[2] # store the index of the current node
250             parent_index = node[3] # store the parent index list of current node
251
252             node_dist = math.sqrt((node[4][0]-self.x_goal)**2 + (node[4][1]-self.y_goal)**2) # calculate the
distance between the current node and goal node
253             if node_dist < 5: # if the node is goal position, exit the loop
254                 self.get_logger().info("Path Successfully Calculated, Moving the robot ...")
255                 break
256
257             for action_set in action_lists:
258                 point, new_heading, tc, c2c= actionnnn(node,action_set[0],action_set[1])
259                 if point not in obstacle_set and point not in closed_set and 0<=point[0]<600 and 0<=point[1]<200:
# check if the new node is in the obstacle set or visited list
260                     x = int(point[0]) # get the x coordinate of
the new node
261                     y = int(point[1]) # get the y coordinate of
the new node
262                     try:

```

```

263         if tc < tc_node_grid[x][y]:                # check if the new cost to
come is less than original cost to come
264             new_parent_index = parent_index.copy()    # copy the parent index
list of the current node
265             new_parent_index.append(index)            # Append the current node's
index to the new node's parent index list
266             new_index += 1                            # increment the index
267             tc_node_grid[x][y] = tc                  # Update the new total cost
268             c2c_node_grid[x][y] = c2c                # Update the new cost to
come
269             new_node = (tc, c2c, new_index, new_parent_index, (x,y), new_heading) # create the new
node
270             hq.heappush(open_list, new_node)          # push the new node to the
open list
271     except:
272         pass
273
274     path = node[3]
275     self.waypoints = []
276     for index in path:                                # loop to mark the path
277         coord = visited[index]                        # get the coordinates of the node
278         x = (coord[0] - 50) / 100
279         y = (coord[1] - 100) / 100
280         self.waypoints.append((x, y))
281     self.waypoints.append(((self.x_goal - 50) / 100, (self.y_goal - 100) / 100))
282
283
284
285
286 def main(args=None):
287
288     rclpy.init(args=args) # Initialize the ROS client library
289     node = AStarControllerNode() # Create an instance of the AStarControllerNode
290     try:
291         rclpy.spin(node)          # Run the node
292     except KeyboardInterrupt:
293         node.get_logger().error("KeyboardInterrupt received!") # Print the Keyboard interrupt message
294     finally:
295         node.destroy_node()      # Destroy the node
296         rclpy.shutdown()         # Shutdown the ROS client library
297
298
299 if __name__ == '__main__':
300     main()

```

